

大家多年打磨的 UI 自动化是否真实提效了？

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

QCon
全球软件开发大会

小红书 GUI Agent 在智能化测试中的工程落地实践

像做 AI Coding 一样做自动化测试

冬司（张亦驰）

小红书 · GUI Agent 方向负责人

目录

- 01 业务背景与痛点
- 02 整体方案设计
- 03 执行引擎 Code as Action
- 04 知识库与主动学习
- 05 总结与展望

01

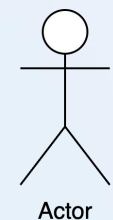
业务背景与痛点

Why We Started — The Pain of Traditional Test Automation



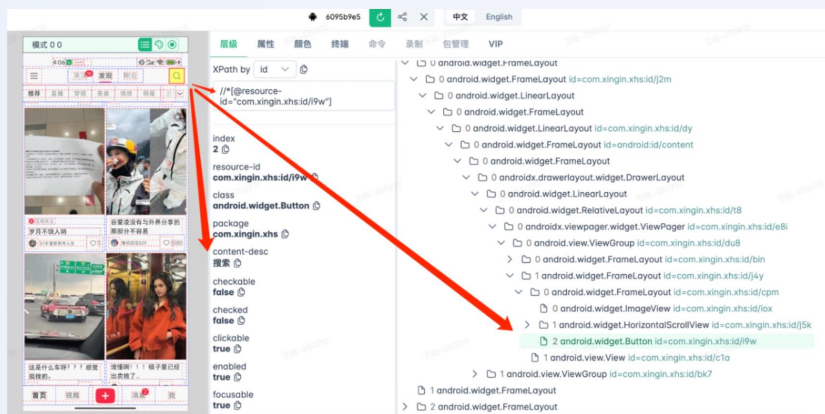
传统 UI 自动化的结构性困境

维护成本极高



定位
XPath

Android



开发
Python

```
class Handy(Runner):
    def setUp(self):
        agent = Agent(device_serial='00008110-000605103609401E', platform=Platform.IOS)

        ele = MultiPlatformRedElement(semantic='自然语义', text='文本', elements={
            Platform.ANDROID: RedElement(
                dom_ele=AndroidElement(descriptionStartsWith='视频',
                    visual_ele=KVisualElement(pic_path_relative='pics/hon_发布加号.png')
            ),
            Platform.IOS: RedElement(
                dom_ele=IOSElement(xpath="//XCUIElementTypeButton[@name='菜单']"),
                visual_ele=AirtestVisualElement(pic_path_relative='pics/hon_发布加号.png', record_pos=
                    (-0.386, -0.507),
                    resolution=(1080, 2340))
            ),
        })
        agent.tap_element(ele, priority=AgentPriority.VISUAL_ONLY)
        pass

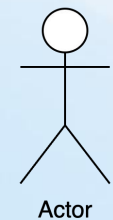
    def test_0(self):
        pass

    def test_0(self):
        pass
```

云测平台执行

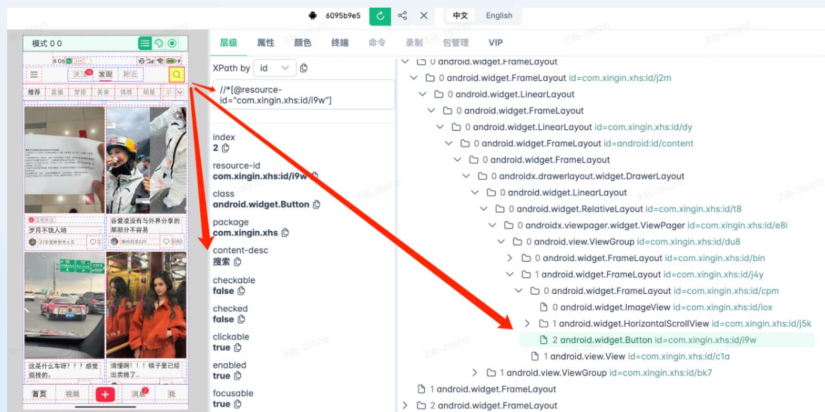
RedMTC

iphone 12
iphone 14
xiaomi 13
oppo find x8
oppo find x7
.....



定位
XPath

iOS



开发
Python

```
class Handy(Runner):
    def setUp(self):
        agent = Agent(device_serial='00008110-000605103609401E', platform=Platform.IOS)

        ele = MultiPlatformRedElement(semantic='自然语义', text='文本', elements={
            Platform.ANDROID: RedElement(
                dom_ele=AndroidElement(descriptionStartsWith='视频',
                    visual_ele=KVisualElement(pic_path_relative='pics/hon_发布加号.png')
            ),
            Platform.IOS: RedElement(
                dom_ele=IOSElement(xpath="//XCUIElementTypeButton[@name='菜单']"),
                visual_ele=AirtestVisualElement(pic_path_relative='pics/hon_发布加号.png', record_pos=
                    (-0.386, -0.507),
                    resolution=(1080, 2340))
            ),
        })
        agent.tap_element(ele, priority=AgentPriority.VISUAL_ONLY)
        pass

    def test_0(self):
        pass

    def test_0(self):
        pass
```

云测平台执行

RedMTC

iphone 12
iphone 14
.....

困境一：自动化脚本维护难



先定位元素 → 再写代码 → 再调试。一个点击操作背后是抓坐标、写 XPath、调试三步，**半小时**起步

时间成本高



iOS / Android / 鸿蒙控件和坐标完全不一致，同一功能三端三套脚本，维护量 ×3

跨端成本 ×3



XPath 是静态的但版本一直在变，小红书每周一版——结构性矛盾

维护成本失控



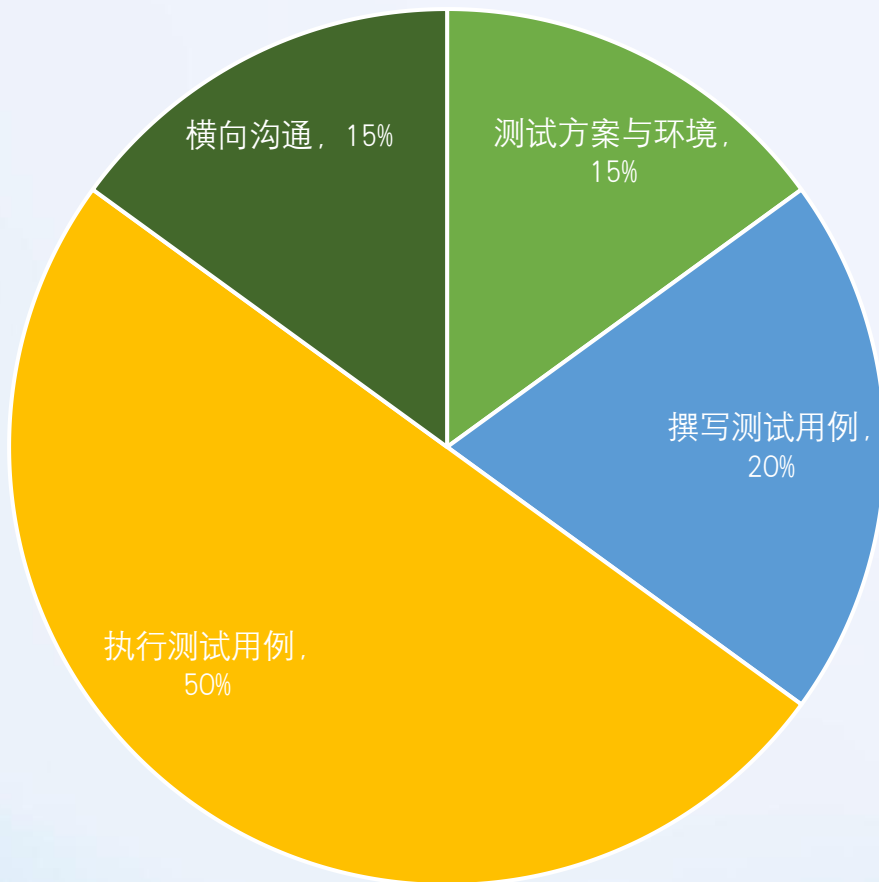
传统断言只能做简单的元素存在性判断，无法理解语义和视觉内容

断言能力弱

```
# 10+ 行硬编码，强依赖机型/坐标/XPath
if self.market_name in ("iPhone 14 Pro"):
    loc = [314, 54]
elif self.market_name in ("iPhone 15"):
    loc = [328, 58]
self.driver.tap(loc)
time.sleep(2)
self.driver.find_element(
    AppiumBy.XPATH,
    '//XCUIElementTypeButton[@name="购物"]'
).click()
search_box = self.driver.find_element(
    AppiumBy.XPATH,
    '//XCUIElementTypeSearchField'
)
search_box.send_keys("卫衣女")
```

● 困境二：用例执行只占一部分

测试投入时间分布：



- 执行用例占一大半，但也只有 50%
- 测试方案和用例生成加起来占了 35%



自然语言表达力的优势

自动化Python代码

```
# 10+ 行硬编码, 强依赖机型/坐标/XPath
if self.market_name in ("iPhone 14 Pro"):
    loc = [314, 54]
elif self.market_name in ("iPhone 15"):
    loc = [328, 58]
self.driver.tap(loc)
time.sleep(2)
self.driver.find_element(
    AppiumBy.XPATH,
    '//XCUIElementTypeButton[@name="购物"]'
).click()
search_box = self.driver.find_element(
    AppiumBy.XPATH,
    '//XCUIElementTypeSearchField'
)
search_box.send_keys("卫衣女")
```

自然语言用例

- 点击左下角我这个tab, 进入个人页
- 进入购物页面
- 搜索卫衣女
- 断言结果页包含商品卡片、价格、大促 icon

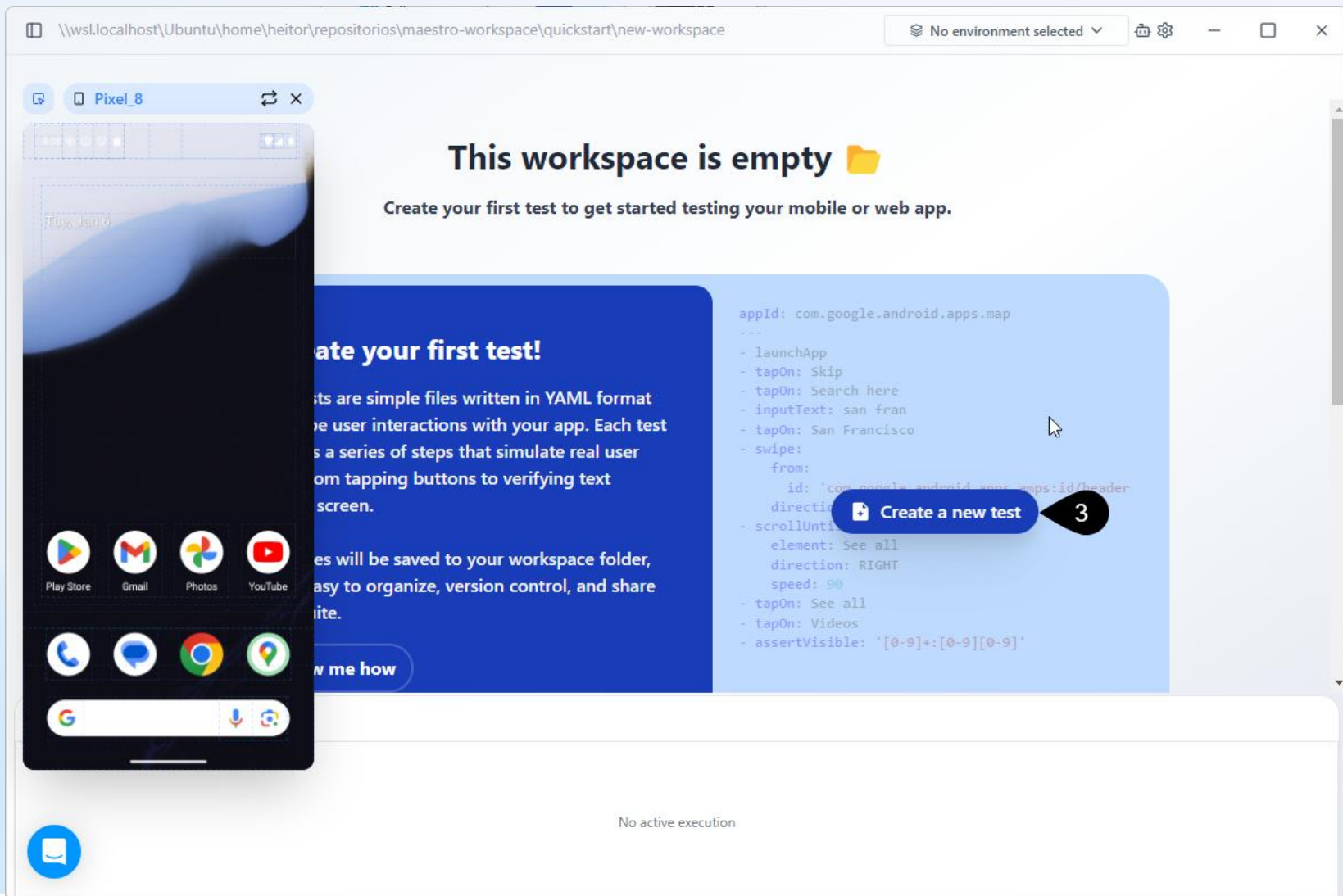


自动化测试的演进的三个阶段





阶段一：Maestro



- 重点在录制生成
- 纯工程脚本
- 维护&更新困难

阶段二：Midscene.js

```
// 👁️ Directly open ebay.com webpage (recommended approach)
await page.launch('https://ebay.com');
await sleep(3000);

// 👁️ Enter keywords and perform search
await agent.aiAct('Search for "Headphones"');

// 👁️ Wait for loading to complete
await agent.aiWaitFor('At least one headphone product is displayed on the page');
// Or you can use a simple sleep:
// await sleep(5000);

// 👁️ Understand page content and extract data
const items = await agent.aiQuery(
  '{itemTitle: string, price: Number}[], find product titles and prices in the list',
);
console.log('Headphone product information', items);

// 👁️ Use AI assertion
await agent.aiAssert('Multiple headphone products are displayed on the interface');
```

- 解决了原子动作泛化
- 解耦掉具体的环境与设备
- 用例生成和维护依然需要大量人工参与

New 使用 Midscene Skills 控制任意平台 | 了解更多

Midscene.js

AI 驱动、视觉感知，适用于全平台的 UI 自动化。

12k+

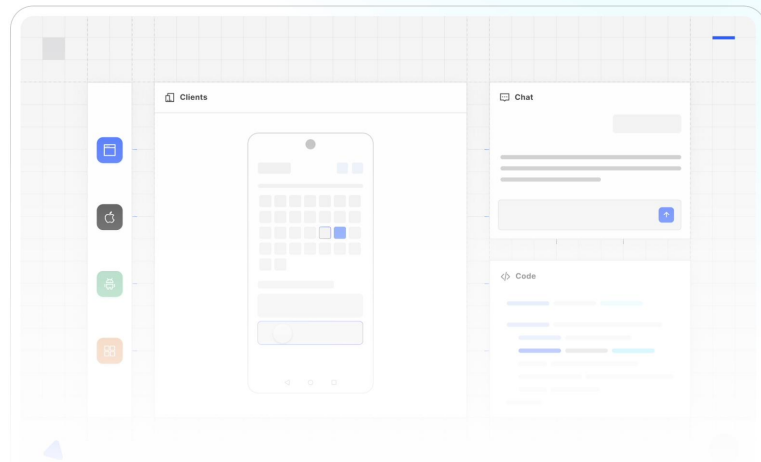
Github Stars

#2

Github 趋势榜第 2 名

使用文档

案例展示



阶段三 探索与泛化 —— 探索与局部落地

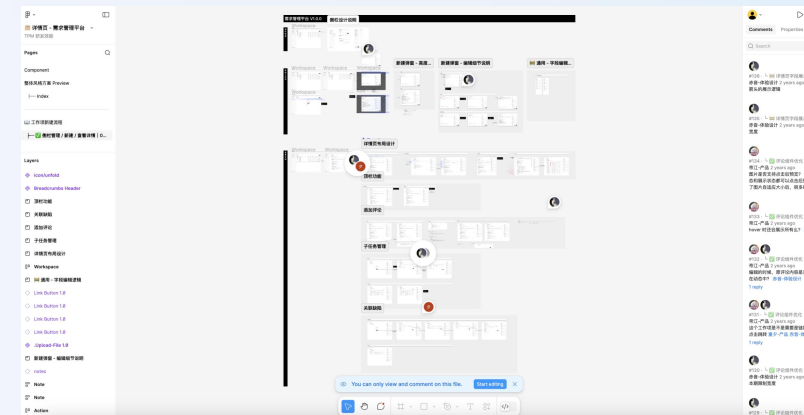
提测文档



PRD



设计稿



Agent 自主探索 + 自我学习，根据需求与代码变更创建与构造测试用例



实战验证 — 春节大促全量落地

26年春节大促期间，GUI Agent 在全业务线中实现规模化落地

自动化执行

75%

自动化率

106 × 428

设备种类 × 场景组合

68%

兼容性 Bug 自动召回率

12.3w+

累计自动化执行用例/次

AI 用例生成

92%

用例采纳率

66%

需求覆盖率

63%

用例渗透率

核心场景固化

600+

S0 核心 Case 常态化运行

人工平替

版本回归测试

零人工

DSL 免发版项目测试

02

方案整体设计

Architecture Overview



可自动化用例分布

70-80%
可自动化

功能回归、流程验证、多机型兼容性检查、接口联动验证、DSL 验证

容易被模型解决

15-20%
可辅助自动化

偶现问题、复杂业务逻辑组合 bug、多端联动场景
(App 下单→后台发货→查看物流)

虽然这些非常规用例占有所有“用例数”的**极少数**，但是这些非常规缺陷占 bug 数的大多数

5-10%

短期无法自动化

极端用户路径、暴力回归、物理设备特殊状态（弱网、低电量）、视觉审美判断

依赖个人品味、大量业务经验、洞察力的疑难缺陷目前来看大模型很难发现

策略：达到 80% 的普通测试水准，而不是如何取代**最顶尖的测试专家**



关键问题与解法对应

脚本执行：成本、泛化

1

执行过程泛化

→ Code-as-Action + 自然语言 SDK

2

模型执行成本高

→ 脚本固化 · 零 Token 回归

3

灵活性 vs 确定性

→ 三层架构分离关注点，确定性前置

上下文管理：需求理解、去幻觉

4

需求理解困难，窗口大小有限

→ 分层知识库 · 按需检索

5

经验性测试，暴力边界用例

→ 自我学习 · 知识沉淀

6

模型迷失/跑偏

→ 路径引导 · 操作图谱

三层架构设计

业务意图层

人类编写 · 版本控制

结构化自然语言描述测试意图，人类唯一需要深度参与的部分

前置 review，意图确认前置

Agent 探索层

LLM 实时执行 · 按需触发

Agent 与 App 实时交互探索，用于初始生成和自修复诊断，生成详细测试计划、测试用例

要求不修改用例意图的基础上探索，只允许泛化用例

可执行代码层

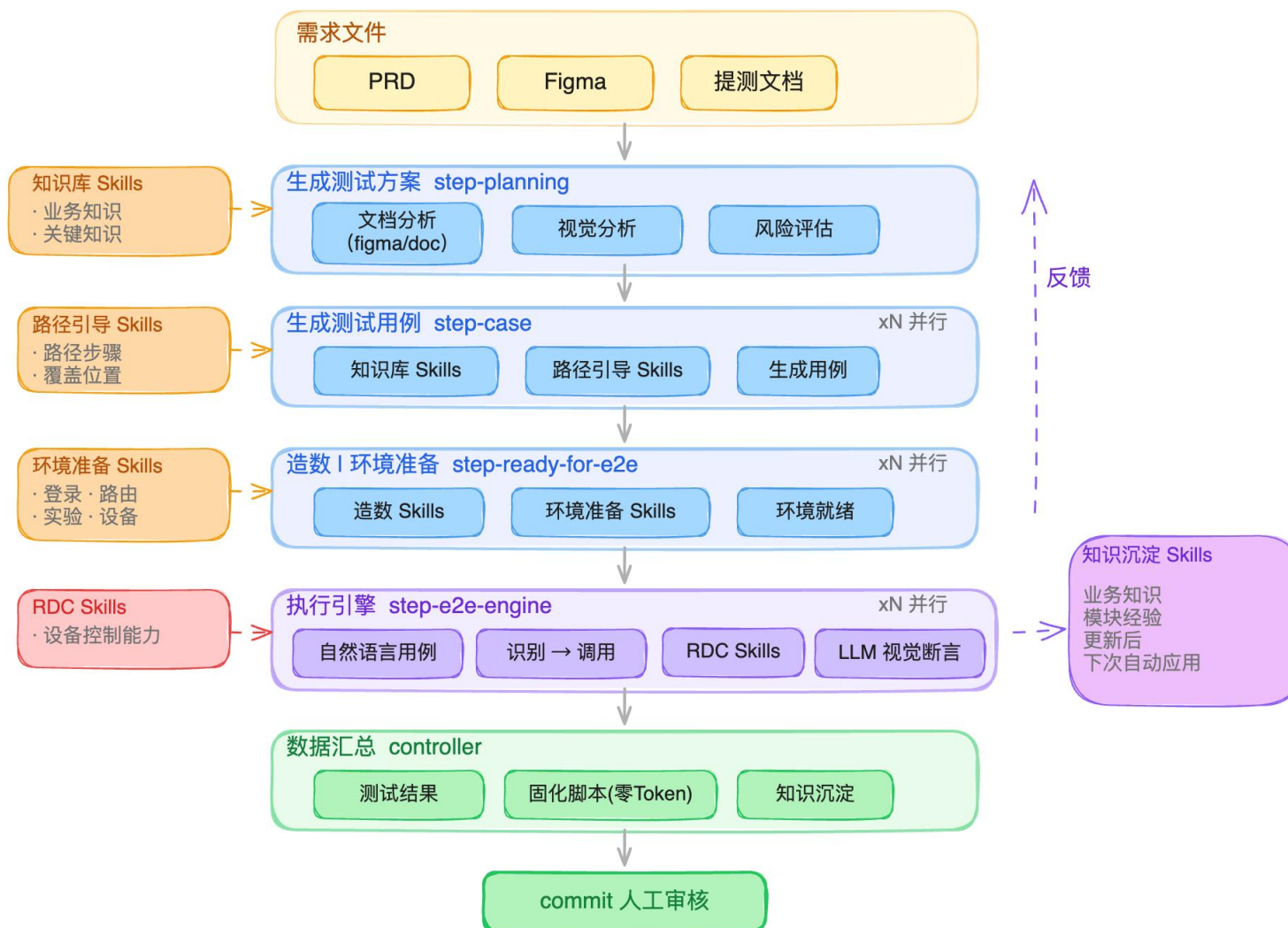
LLM 生成 · CI 回归

确定性测试代码日常 CI 回归，“零”Token 成本

要求确定性 > 95% 只允许局部纠偏

小红书 GUI Robot 技术架构

UI Robot 整体架构



03

执行引擎

智能规划与生成

执行引擎-三个关键问题

1

执行过程泛化

→ Code-as-Action + 自然语言 SDK

环境错误/执行错误/用例错误/歧义/模型出错

2

模型执行成本高

→ 脚本固化 · 分离视觉执行器

视觉模型和 opus 太贵

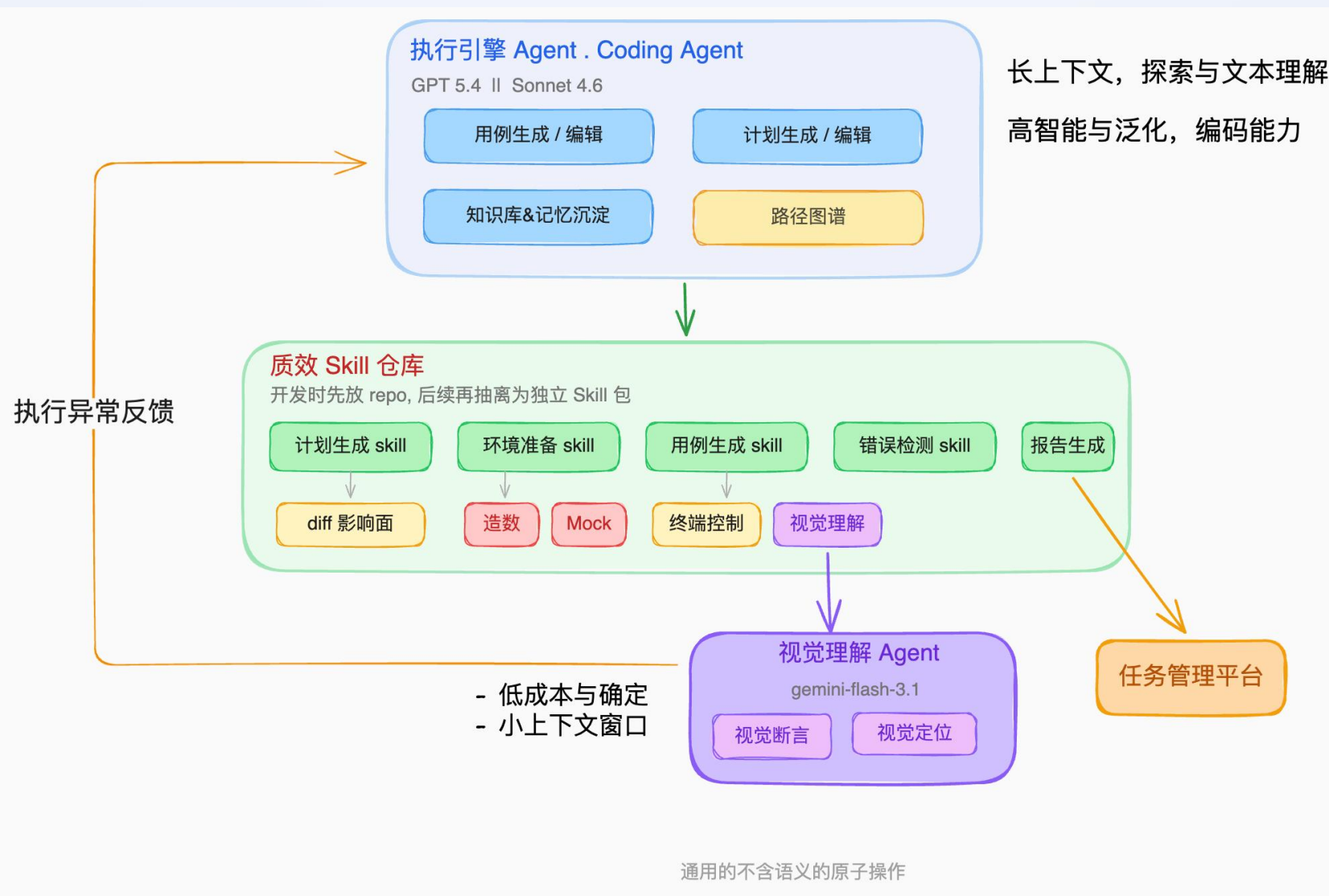
3

灵活性 vs 确定性

→ 三层架构分离关注点，确定性前置

意外的弹窗要不要自己关，算不算 bug?
设计稿和 PRD 冲突，怎么算对?

执行引擎-整体架构





执行引擎-视觉识别 skill

元素定位

语义理解 + DOM 结构 + 视觉识别三层定位方案,
抹平端差异

统一驱动

统一操作 API 封装三端 (Android/iOS/鸿蒙)
Driver 差异

调用:

```
python3 scripts/locate_element.py \  
    --user-query "'确认退出'按钮" \  
    --image-path "/path/to/screenshot.png" \  
    --agent-id "7d638c5750e54fb9bcc02d3b11272533" \  
 \  
    --app-name "小红书" \  
    --app-version "9.12.0" \  
    --device-platform android \  
    --device-serial "10AD7L1FKM001YS" \  
    --device-x 1080 \  
    --device-y 2388
```

返回:

```
{  
  "success": true,  
  "center": [723, 1337],  
  "leftTop": [555, 1263],  
  "rightBottom": [891, 1411],  
  "reason": "在页面中心的'确定要退出登录吗?'确认弹窗中,  
识别到位于右侧的黑色圆角矩形按钮, 其内部包含'确认退出'文字。",  
  "confidenceScore": null  
}
```



执行引擎-视觉断言 skill

指令预处理

动态识别弹窗和业务分支，根据执行结果实时更新指令序列；

结果断言

多模态理解页面截图 → 语义断言 → 输出断言结论 + 置信度

调用：

```
python3 scripts/assert_condition.py \  
    --app-name "小红书" \  
    --app-version "9.19.3" \  
    --device-serial "00008110-00117884223A401E" \  
    --device-platform ios \  
    --user-query "屏幕左上角展示一个图标，并且这个图标不是三道横杠的样式" \  
    --image-path "/path/to/screenshot.png" \  
    --agent-id "08d9395243924d10b0960bf011aacc6d"
```

返回：

```
{  
  "success": true,  
  "assertResult": true,  
  "assertDesc": "图片左上角确实存在一个图标，该图标是一个不完整的圆形，不是三道横杠的样式，符合描述。"  
}
```



执行引擎-终端控制 skill

终端控制

类别	能力	说明	Android	iOS
设备连接	connect(deviceId)	连接指定设备	ADB	WDA
设备连接	disconnect()	断开连接	✓	✓
画面获取	screenshot()	截取当前屏幕	screencap	WDA screenshot
基础操作	click(x, y)	点击坐标	input tap	WDA tap
基础操作	swipe(x1,y1,x2,y2,dur)	滑动	input swipe	WDA swipe
基础操作	input(text)	输入文本	input text	WDA type
系统控制	press(key)	按键操作	keyevent	WDA press
系统控制	home()	Home 键	KEYCODE_HOME	pressHome
系统控制	back()	Back 键	KEYCODE_BACK	无(需实现)
应用管理	launch(pkg)	启动应用	am start	WDA launch
应用管理	kill(pkg)	杀应用	am force-stop	终止进程
应用管理	install(apk/ipa)	安装包	pm install	ios-deploy
应用管理	uninstall(pkg)	卸载应用	pm uninstall	删除应用
设备信息	getDeviceInfo()	设备信息	getprop	WDA info
设备信息	getScreenSize()	屏幕尺寸	wm size	WDA rect
设备信息	getBattery()	电量信息	dumpsys battery	WDA battery
....				

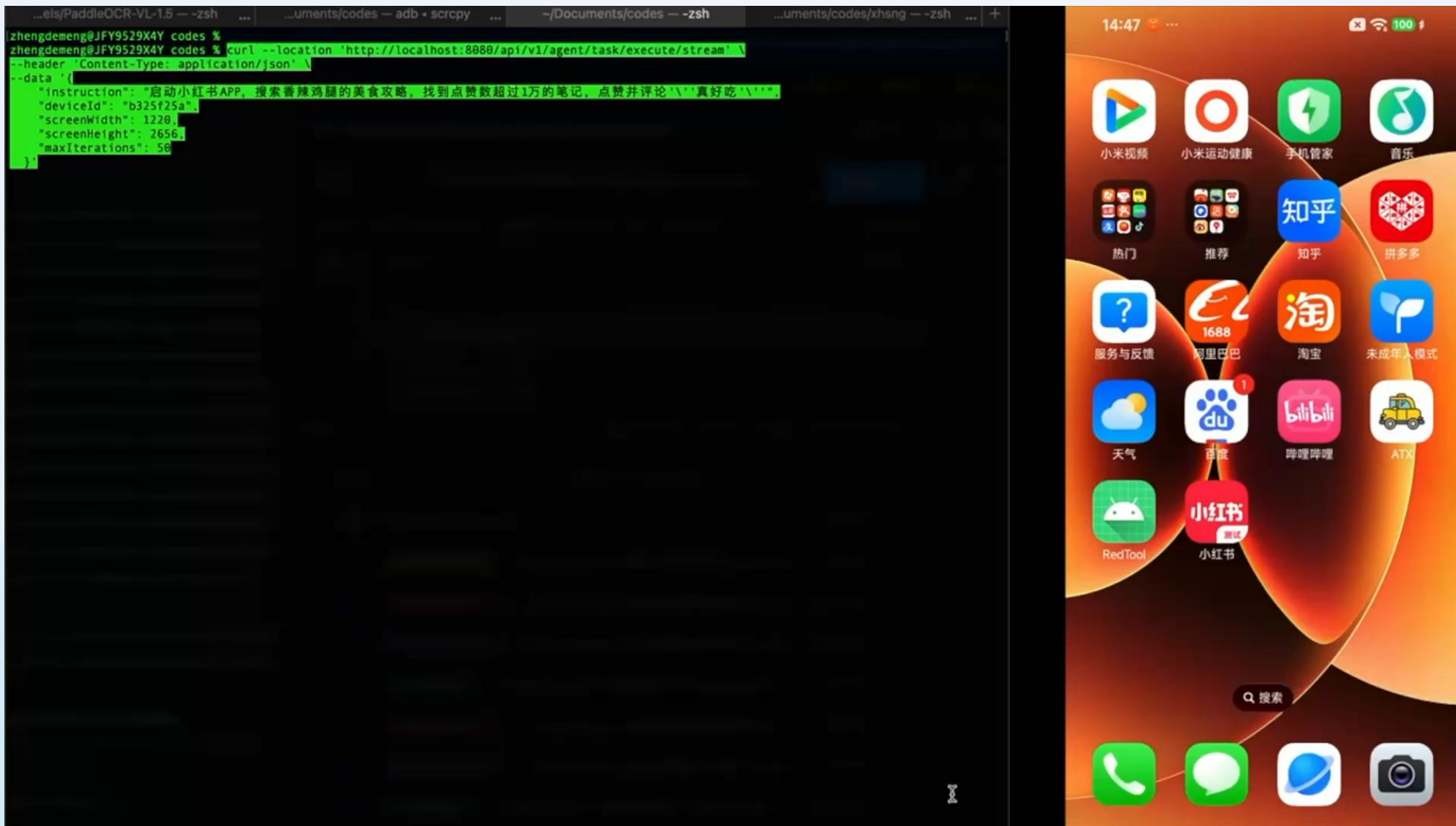
环境准备

最简单又最难坑

- 应用信息获取 (userId, deviceId)
- XHS登录能力
- 跳转页面
- 埋点获取
- 已支持的可测性构造测试环境
 - 实验Mock
 - Switch Mock
 - 抓包 Mock
 - 环境切换
 - 模拟定位
 - 下载RN包
 - DSL配置
 - 手势轨迹 (iOS)
 - 降级
 - 内测
 -



执行引擎-自然语言控制能力演示



“启动小红书APP, 搜索香辣鸡腿的美食攻略, 找到点赞数超过1万的笔记, 点赞并评论”“真好吃!”



执行引擎-纯视觉模型能力的限制

断言失败 case:
页面内容为竹子

← 上一步 33 / 33 下一步 →

步骤详情

操作: 断言

描述: {"memory": false, "objectsRelation": "ALL", "targets": [{"expectation": "页面为竹子的详细介绍页, 左上角有返回按钮"}]}

时间: 2026-03-26 18:48:45

断言: ✖ 失败
图片中的页面标题是'竹子', 但详细介绍的植物是'玉簪植物', 并非竹子。虽然左上角有返回按钮, 但页面内容与预期不符。

图片: [查看原图](#)

用例原文

{"memory": false, "objectsRelation": "ALL", "targets": [{"expectation": "页面为竹子的详细介绍页, 左上角有返回按钮"}]}

Reason

{"assertResult": false, "assertDesc": "图片中的页面标题是'竹子', 但详细介绍的植物是'玉簪植物', 并非竹子。虽然左上角有返回按钮, 但页面内容与预期不符。"}]

18:48 RedJump

<

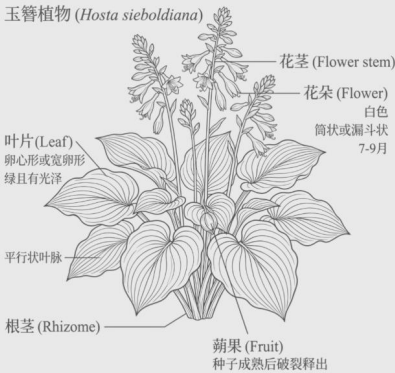


竹子

August Lily

目 天门冬目 Asparagales
科 天门冬科 Asparagaceae
属 玉簪属 Hosta
种 Hosta sieboldiana

玉簪植物 (*Hosta sieboldiana*)



根茎与叶片

具明显的根茎, 叶丛生, 叶片 卵心形或宽卵形; 叶色绿且有光泽, 叶表面光滑, 叶脉呈平行状

花序与花朵

执行引擎-当前视觉模型的局限

1. 业务不理解
2. 精度与智能程度不够
3. 受到预训练数据的影响

计划生成

```
"operation": {
  "action": "点击",
  "type": "",
  "actionParam": "",
  "targets": [
    {
      "ancestor": "商城首页",
      "anchor": "推荐Tab",
      "name": "推荐Tab右侧任意商品分类Tab",
      "position": "分类栏",
      "anchor_relation": "右侧",
      "desc": "商品分类标签页, 如穿搭Tab, 不是春上新
Tab"}
    ]
  }
}
```

```
"assert": {
  "vision": {
    "memory": true,
    "objectsRelation": "ALL",
    "targets": [
      {
        "expectation": "切换至相邻右侧分类页, 分类Tab文案
为粗黑字体 Tab下方展示三行二级分类图片和文案, 二级分类图片和
二级分类文案一一对应, 无空白三级分类图片, 三行二级分类图片下方
展示双列商品列表, 商品信息展示商品图、商品名称、价格优惠标签或
权益标签、售价, 商品列表没有空白图片"
      }
    ]
  }
}
```



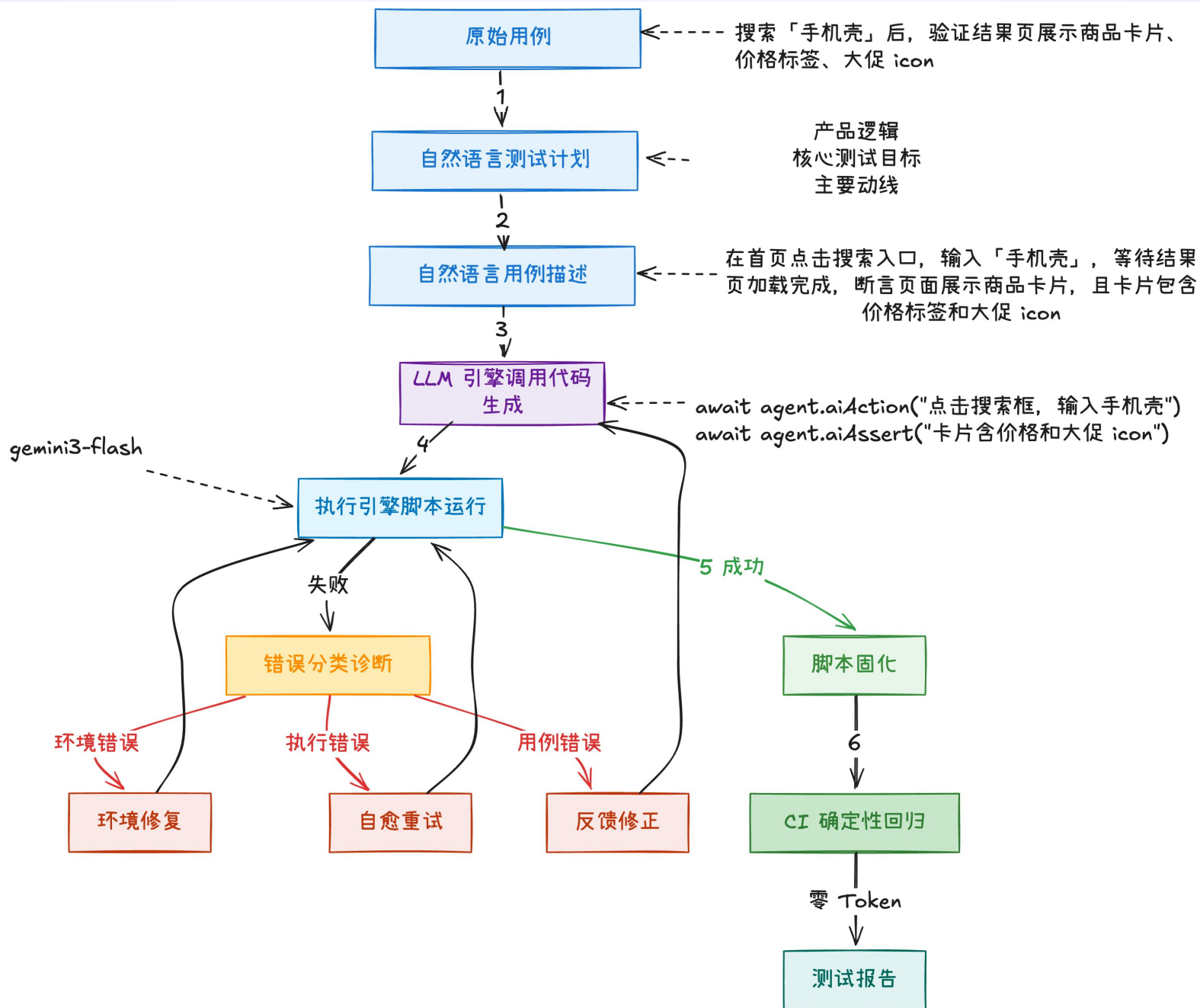
计划生成-代码 vs ToolCall

Code-as-Action vs ToolCall 核心对比

对比维度	ToolCall	Code-as-Action	结论
可固化性	每次都要调 LLM 无法沉淀为资产	验证通过 → 固化为代码 CI 零 Token 回归	Code 胜出
调试能力	黑盒调用 只能看日志排查	生成标准代码 IDE 单步调试 + Review	Code 胜出
表达力	受限于预定义工具集 无法组合复杂逻辑	代码表达无上限 支持自定义断言	Code 胜出
执行成本	每次回归都消耗 Token 每做一个行为就是一次模型请求	固化后零 Token 执行 一次能生成 N 个行为动作，可编程	Code 胜出
遵循性	训练到模型内	依赖模型的 coding 能力	ToolCall 胜出



计划生成-





固化后脚本示意

```
#!/usr/bin/env bash
# TC-003 回归测试 - 个人页历史对话Tab基础功能 (P0)
# 用法: bash tc003_regression.sh --serial <device_serial>

# — Step 1: path-guide - 检索导航路径 + 坐标 —————
node path-guide/cli.mjs \
  --workspace workspaces/diandian.json \
  --requirement "进入点点个人页, 切换历史对话Tab, 点击历史对话卡片" \
  --output "$OUTPUT_DIR/route.json"

# 从返回结果提取元素坐标 (置信度 100%, 路径已命中图谱)
AVATAR_X=$(python3 -c "import json; ...") # → 9
TAB_X=$(python3 -c "import json; ...")    # → 35
TAB_Y=$(python3 -c "import json; ...")    # → 41

# — Step 2: rdc-mobile-use - 控制设备执行 —————
bash rdc-mobile-use/scripts/task_execute_default.sh \
  --serial "$DEVICE_SERIAL" \
  --cmd "{{ATX}}" app-open --package com.liveverse.diandian" && \
  --cmd "{{ATX}}" tap --percent $AVATAR_X,9" && \
  --cmd "{{ATX}}" tap --percent $TAB_X,$TAB_Y" && \
  --cmd "{{ATX}}" screenshot --output $OUTPUT_DIR/evidence/tab.png" && \
  --cmd "{{ATX}}" tap --percent 50,57" && \
  --cmd "{{ATX}}" screenshot --output $OUTPUT_DIR/evidence/card.png"

# — Step 3: 断言 - OCR 验证截图 —————
bash rdc-mobile-use/scripts/task_execute_atomic.sh --quick \
  --serial "$DEVICE_SERIAL" \
  --exec "{{ATX}}" ocr --image $OUTPUT_DIR/evidence/tab.png --contains '历史对话'"

bash rdc-mobile-use/scripts/task_execute_atomic.sh --quick \
  --serial "$DEVICE_SERIAL" \
  --exec "{{ATX}}" ocr --image $OUTPUT_DIR/evidence/card.png --contains '对话'"

echo "TC-003 PASS - evidence: $OUTPUT_DIR/evidence/"
```

用例执行完成之后，自动根据探索过程生成完整的用例执行脚本。



计划生成-用例 Case

TC-002 个人页框架基础布局与上滑吸顶行为

P0

Path Guide 86%

知识库 3 docs

基于 PRD 「个人页框架改版+收藏」 自动生成 | 存量路径 4 + 推导路径 1

操作步骤

1 启动 App



2 点击头像 -> 进入【个人页】



3 查看页面初始状态



4 向上滑动内容列表 -> 验证吸顶行为

Android | 无实验 | 无Switch

预期结果（关键断言）

页面初始状态

- 顶部：返回按钮（左）+ 设置按钮（右）
- 中部：用户头像（居中）+ 昵称 + 地理位置
- 下方：历史对话 Tab（默认激活）+ 收藏 Tab

上滑吸顶后

- Tab 区域吸顶固定不滚动
- 个人信息区（昵称/位置）收起
- 仅展示缩小版用户头像

位置兜底：未授权时显示授权入口

核心风险

- 上滑吸顶动画细节 PRD 未描述，需与开发确认吸顶临界点
- 地理位置未授权兜底显示的具体文案需确认

自动化候选

- 验证上滑后 Tab 吸顶状态
- 验证初始页面各元素存在性

测试数据：需要地理位置已授权账号（已授权分支）+ 未授权账号（授权入口兜底分支）

04

知识库与主动学习

Knowledge Management & Memory System



知识库面临的挑战

模型钻进死胡同 -> 路径引导

纯自己探索，不能理解业务，容易跑偏绕远再回来

信息分散 -> 一个大仓

分散在仓库 md、内部文档平台、用例平台、QA 私人知识库

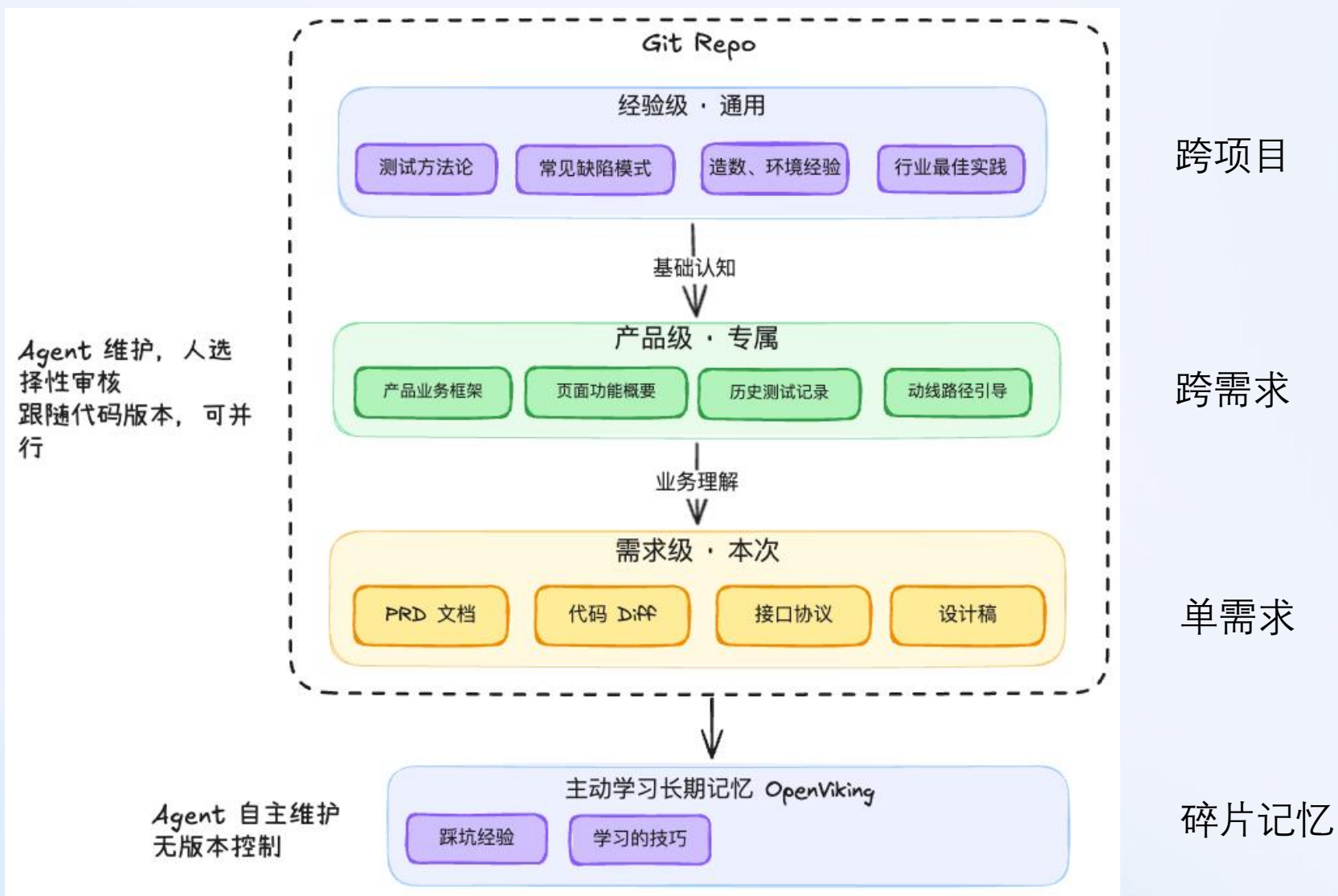
难以沉淀为资产 -> 自动沉淀

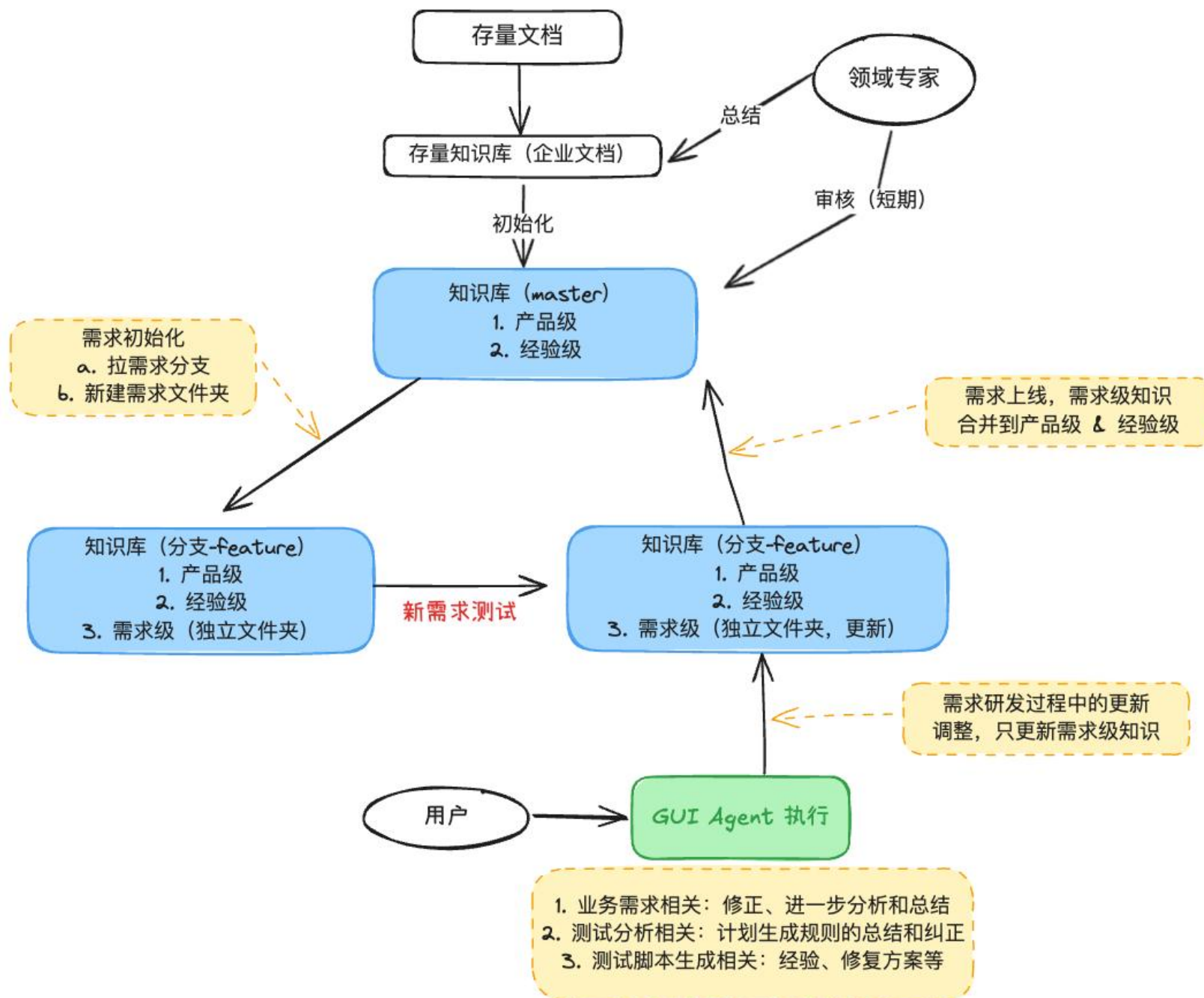
维护知识库成本太高，很多时候不维护只是因为维护太贵

内容过载-> 主动清理

记忆太少重复踩坑，太多产生幻觉

知识库-分级上下文





知识库 - 目录分层标准

L0 摘要层

快速索引，帮助 Agent 判断是否需要深入查看
200 行上限 · 自动淘汰过期内容

L1 概览层

功能模块概览，提供导航方向

2000 行上限，提供本目录下的层级结构，作为路由

L2 详情层

完整的测试知识与经验记录

按需逐层检索 · 置信度阈值过滤

所有上下文都在仓库内，跟代码在一起 — 像 AI Coding 一样管理测试知识

```
— Agents.md
— docs/
  — 产品介绍/
    — Agents.md
    — 产品背景
    — 主要流程
    — 场景介绍
  — 测试计划 plan
    — 测试计划 1/
      — plan.md
    — 测试计划 2/
      — Agents.md
      — 测试用例说明 1
      — 测试用例说明 2
  — e2e-scripts/
    — module1/feature1
    — module2/feature2
    — module3/feature3
  — configs/
  — skills/
  — memory/
    — YYYY-MM-DD.md
    — 2025-04-01.md
    — 2025-04-02.md
    — ...
```

- 本目录的内容摘要与子目录结构说明
- 本目录的内容摘要与子目录结构说明
- 测试计划的上下文与测试脚本内聚
- 生成的测试计划，挑选 e2e
- 本目录的内容摘要与子目录结构说明
- 可执行的 E2E
- 执行配置和临时配置
- 可复用的操作技能
- 长期记忆存储
- 每日记录 (如 2025-04-01.md)



知识库 - 路径引导

UI Explorer | WS: xiaohongshu | Ver: v1.0 | AB: ALL | A | B | [Review](#) | [批量MD](#) | [关联KB](#) | [补位置](#) | [Verify](#) | [Merge](#) | [Gemini](#) | [Export](#) | [Import](#) | [Reset](#)

Knowledge | [Verify](#)

[分析](#) | [历史](#)

进入心愿卡介绍页

▲ 附加上下文 (选填)

↓ DeepLink 起点 (如 app://chat, 精)

🔍 Query 补充 (如「聊天页头像入口」)

Redoc 文档 URL (PRD 链接)

📁 上传截图 (PRD / 设计稿)

[分析路径 →](#) | [JSON](#)

用户通过首页侧边栏进入创作者中心, 在直播开播设置的信息卡管理流程中, 选择并创建心愿卡模板, 最终通过模板库的信息按钮进入心愿卡介绍页。

语义锚点

target: 心愿卡介绍页 | action: click:info_button

✓ 存量路径 (0) | 点击高亮

未找到相关存量路径

💡 推导路径 (1) | 点击展开

通过创作者中心... | 高置信 | 延伸 | 0.9%

[Layout](#) | [Search](#)

[+ Add Screen](#)

路径引导 - 图谱生成

上传截图 (PRD / 设计稿)

分析路径 →

推导路径 (3) 点击展开图

对话流直达个人页路径 95%

Chat → New Personal Profile

个人页功能导航路径 90%

New Personal Profile → Settings

Layout System Settings

上传截图 (PRD / 设计稿)

分析路径 →

知识缺口

state 个人页上滑时的Tab吸顶及头像缩小状态转换
关联: component:tab_sticky

flow 地理位置未授权时的授权入口兜底流程
关联: entity:地理位置

component 收藏Tab的空状态UI表现 (无按钮)

Layout Tence



页面节点概要

“个人页”页面详情

页面别称：个人主页、新个人页、改版独立端个人页

页面入口

- 点击对话流中头像 → 进入个人页（改版后取消侧边栏入口）
- 收藏成功 toast "去看看" → 进入个人页收藏 tab
- 用户反馈渠道入口（设置页中"和开发者聊聊"旁）

deeplink

（暂无，待补充）

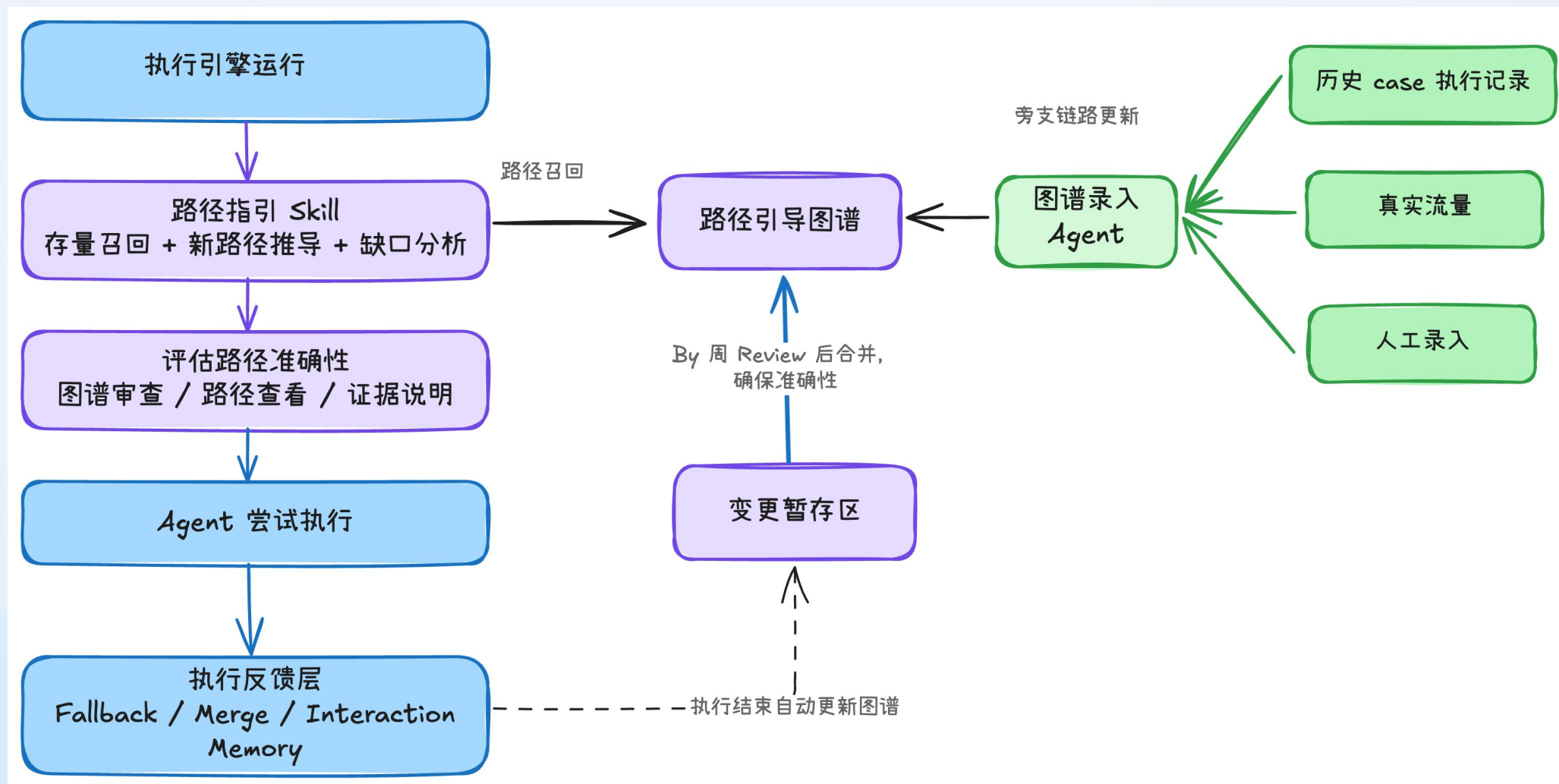
页面详情

整体布局

区域	位置	说明
返回按钮	顶部左侧	单击返回至进入个人页前的页面
设置按钮	顶部右侧	单击进入设置页，设置页内容继承线上所有选项
用户头像	顶部居中（默认态）	单击跳转头像详情

- 触发路线
- deeplink
- 布局结构
- 核心动线

路径引导流程



路径引导思路

字段	含义	怎么用
recalledPaths	图谱中已有的精确路径	优先使用，每个 node 有截图和操作
suggestedPaths	图谱没有、AI 推导的路径	次选，步骤加 待验证
missingKnowledge	图谱空白项	在用例里标注，反馈给图谱维护同学
confidence.overall	整体置信度 0-100	< 60 要人工 review

存量召回：100%
增量召回：75%
高置信度占比：90%

confidence	处理
≥ 80	直接用，不用特殊处理
60-79	用，用例里注明"建议执行后校验"
< 60	加 ⚠ 警告，建议人工 review
recalledPaths 空	降级用 suggestedPaths，步骤全部标 ⚠
文件不存在	跳过路径指引，不要阻塞，在文档头说明原因

05

总结与展望

Knowledge Management & Memory System

关键成果

- 对于新需求，增量节点的推导，用例覆盖 75%
- 存量需求路径覆盖 95%
- 用例执行稳定性 98%
- 单用例执行成本 1\$
- 回归推理不消耗 Token

总结

- **Code-as-Action**: 代码生成比 ToolCall 表达能力更好、容易变更、也更经济
- **脚本固化**: 日常回归走工程脚本, AI 只在生成/自愈时介入, 但整体一套方案
- **分层知识库**: 像写代码一样管理测试知识, 会话记忆、长期记忆、知识库、知识图谱, 各司其职

几个坑

“

让人来做决策和制定标准，让 Agent 来做执行和探索；不要一开始就试图让 Agent 独自端到端解决所有事情

“

重点不是打造通用 Agent，而是基于 Coding Agent 做 GUI 的 Harness，把 UI 自动化从脆弱脚本升级成可理解、可执行、可验证的智能系统。

“

好的自动化 Agent 不是评测集里迭代出来的，提示词和上下文都是真实场景下跑出来的；不要为短期指标做工程硬编码

未来展望

全业务覆盖

电商场景验证后，同一套知识图谱架构向
搜索、IM、广告铺开
一套方法论，驱动全 App 质量体系

扩大自进化范围

每次执行都是学习机会——故障复盘、对话记录自动入库
知识图谱持续生长，系统越跑越聪明

质量动作前置

Agent 介入需求评审阶段，从读懂 PRD 到生成用例，Testing、Coding 可以是一套上下文
测试不再是研发末端的补丁，而是设计的一部分

QCon
全球软件开发大会

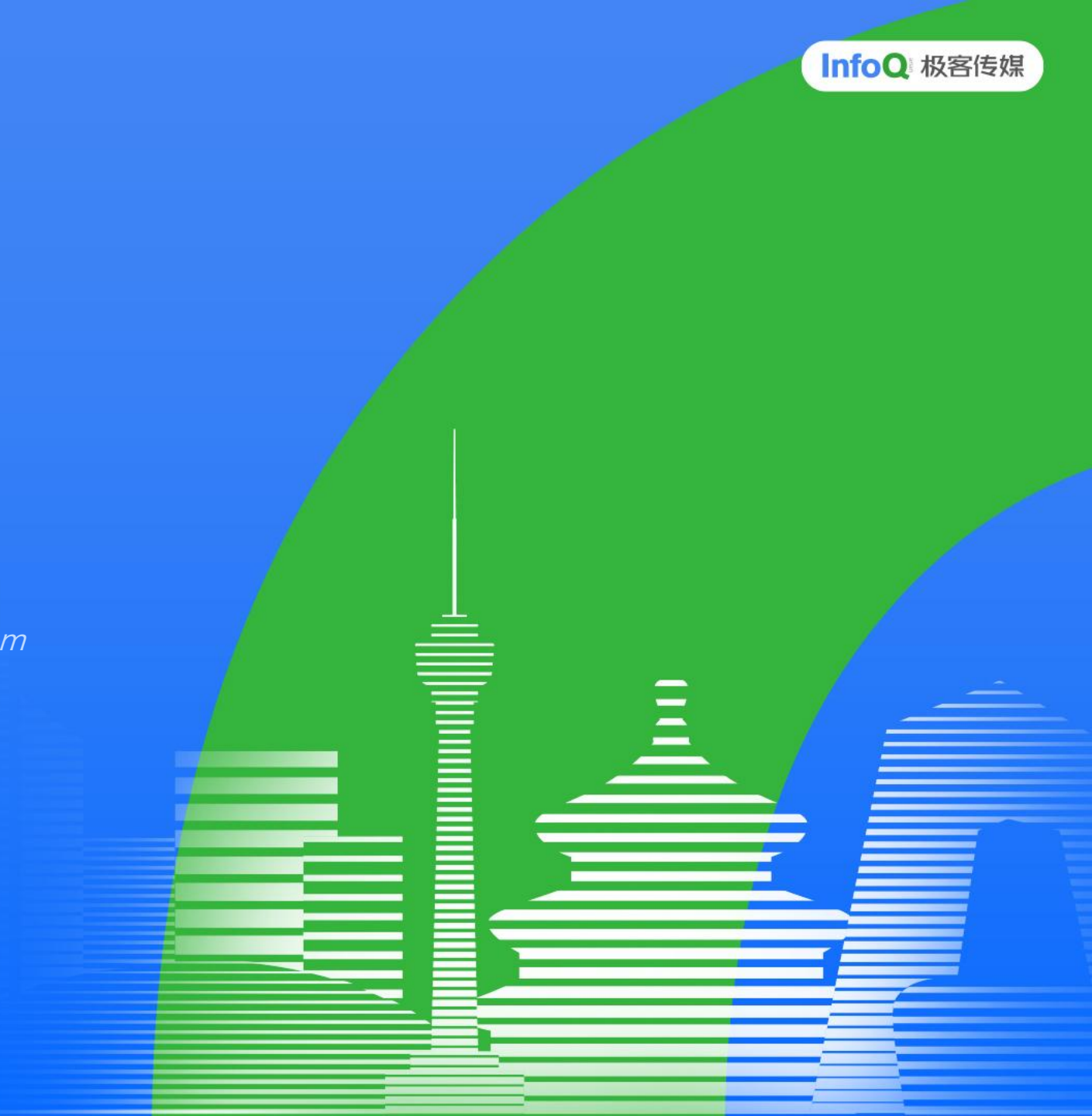
THANKS

让人回到真正需要判断力的地方

Agent is Not Replacing Engineers, It is Freeing Them

冬司（张亦驰） · 小红书

QCon 北京 2026



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

